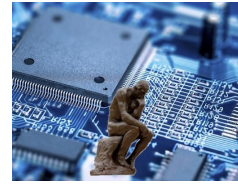




**POLITECNICO
DI TORINO**

Dipartimento di
Elettronica e
Telecomunicazioni



DIGITAL SYSTEMS ELECTRONICS

Academic year 2023-2024

LABORATORY NR. 3

DUE DATE: April 8, 2024
DELIVERY DATE: April 8, 2024

GROUP 03

CONTRIBUTIONS:

- El Amrani Dawid 100%
- Ielmini Giovanni 100%
- Di Carolo Pierfelice 100%
- Pavone Lorenzo 100%

The members of the group listed above declare under their own responsibility that no part of this document has been copied from other documents and that the associated code is original and has been developed expressly for the assigned project.

Contents

This report is relative to the third laboratory experience of this course.
The purpose of this lab session was to learn how to design arithmetic circuits that add, subtract, and multiply binary numbers.

1. 4-bit sequential RCA

a) Design entry:

The circuit under exam receives two signed binary numbers (4 bits) and a clock/reset, and returns as output their sum (a 4-bit vector) and a bit that indicates overflow.

The top level entity is “es1”, and it makes use of a few components:

- “regn”, a register for the inputs and the 4-bit output. We have two of them at the input of the circuit: they save the inputs and pass them to the adder at every rising edge of the clock, or reset the inputs when they are told to. We also have one at the output, and it does the same thing for the 4-bit output of the adder, the sum.
- “flipflop” serves the same purpose, but for a single bit input. We use it to save/reset the overflow output.
- “rca” is the main component, the adder. It adds the bits of the two inputs one by one, using “full_adder” and returns the sum of the two, specifying if there is overflow or not.
- “full_adder” sums two bits and the carry, returning the sum and the new carry.
- “display” receives a 4-bit signed input binary number and prints it in hexadecimal form.

b) Functional simulation:

In the testbench, we assign an arbitrary value to the two inputs, and check if the results (the sum and the overflow) are coherent and follow the clock and reset commands.

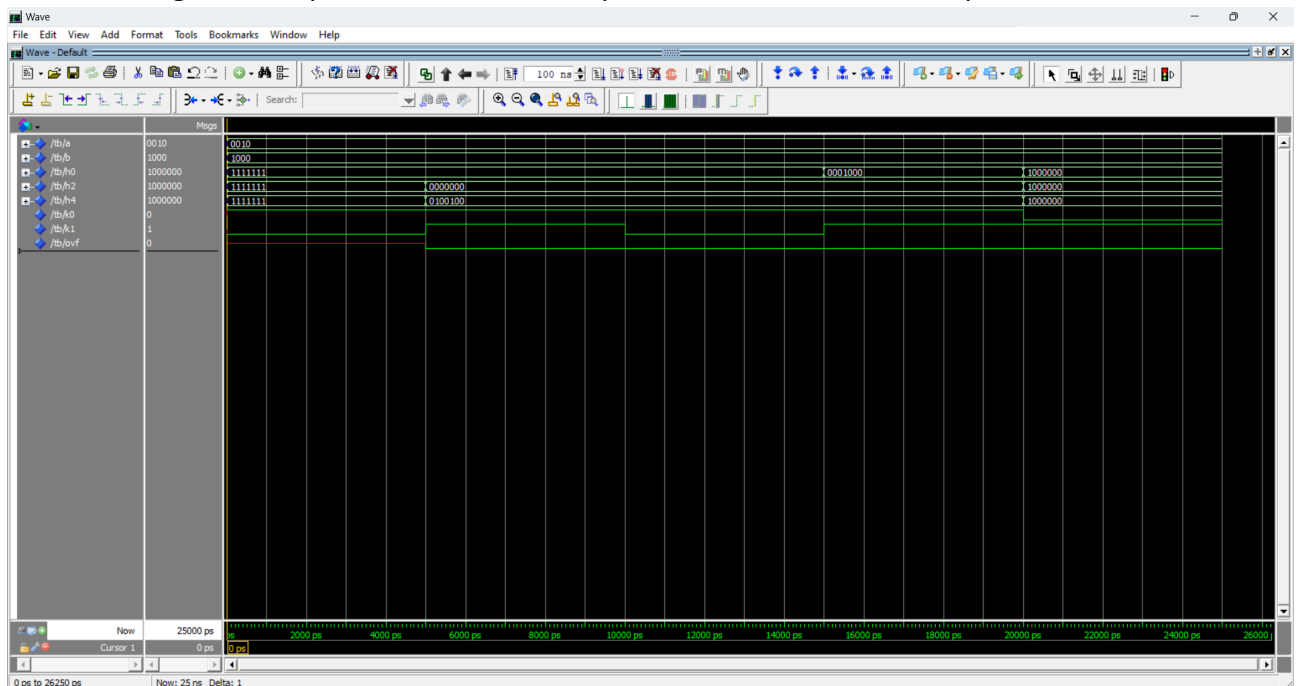
Since the circuit was pretty trivial, we decided to test it only for one case.

c) Synthesis:

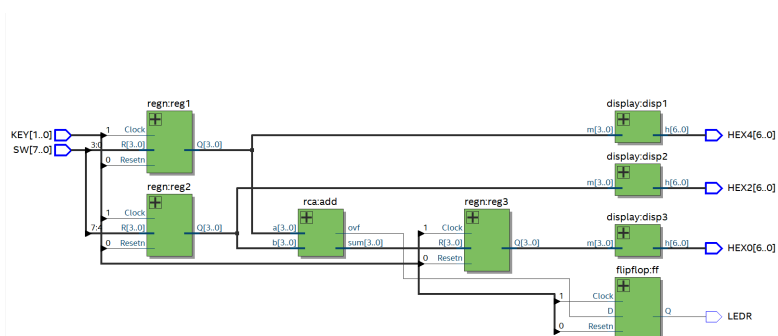
a and b are the input 4-bit binary numbers, h0, h2 and h4 are the three displays, respectively for a, b and the sum, k0 and k1 are the clock and the reset buttons, and ovf represents the overflow.

As we can see from the picture, the displays are initially all off. at the first clock cycle, the two inputs are shown, and at the second clock cycle the output is shown. When the reset command is enabled, all the displays reset to 0.

The delay between representation of the inputs and the representation of the output is due to the registers we placed both on the way in the circuit and on its way out.



For what concerns the timing analyzer, we obtained that the maximum operating frequency of the circuit is 621,12 MHz at 0° C, and 638,16 MHz at 85° C.



The longest path goes from reg1 to flipflop, passing through rca.

2. 4-bit sequential Adder/Subtractor

a) Design entry:

The circuit under exam receives two 4-bit binary numbers (in signed form), a clock/reset and a selector (to decide whether to add or subtract the two input numbers) and returns the result of the operation (a 4-bit signed binary number) and a bit that represents if there is overflow.

The top level entity is “es2”, and it makes use of some components:

- “regn” and “flipflop” have the same purpose as point 1.

- “rca” has been modified with respect to the precedent point: it has one more input, “sel”, that when enabled negates the second input number (“b”). This way, as we call 4 times the component “full_adder”, we can have two possible results: $a+b$ or $a+(-b)$.

After calculating the result of the operation, “rca” even tells us if we have overflow or not.

- “full_adder” is the same as before.
- “display” receives a 4-bit input number and prints it on the seven segment display of the FPGA board, in its hexadecimal form.

b) Functional simulation:

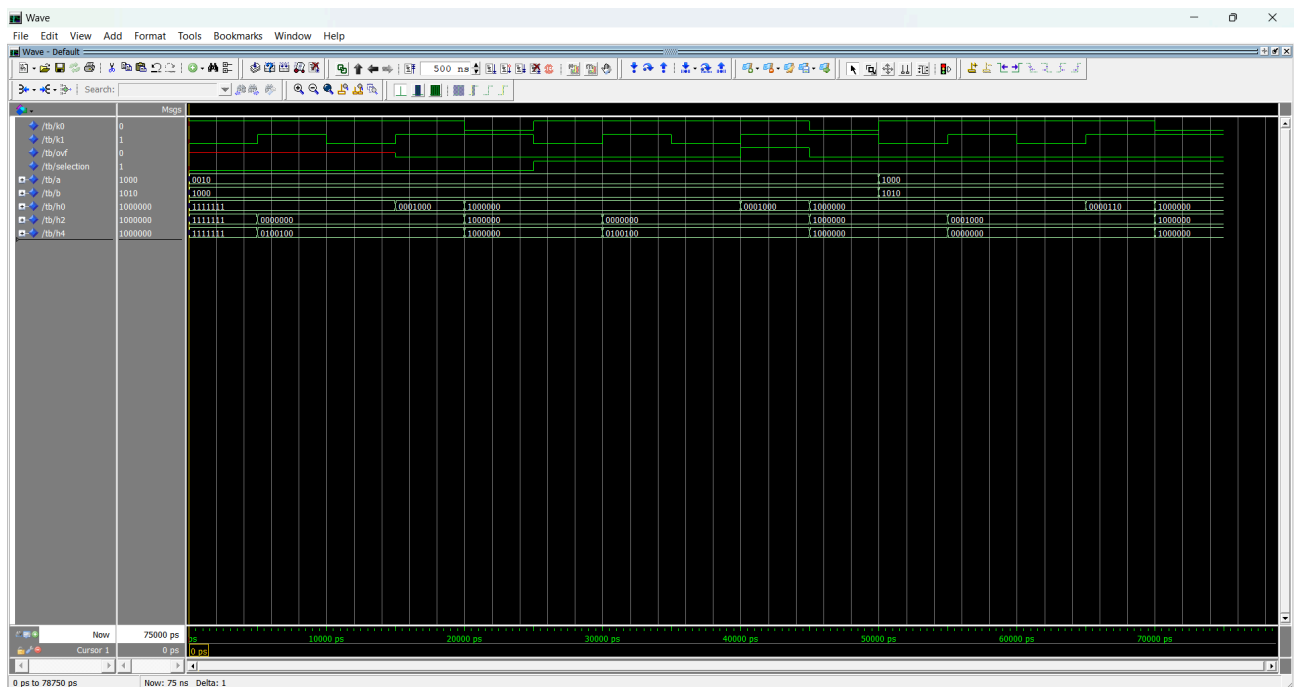
In the testbench, we decided to check if the design was correct, subjecting it to different input scenarios.

We assigned arbitrary values to a and b, trying both to sum them and to subtract one from the other (getting overflow), testing both the functionalities of the circuit, and finally tried with different values for a and b a subtraction without overflow.

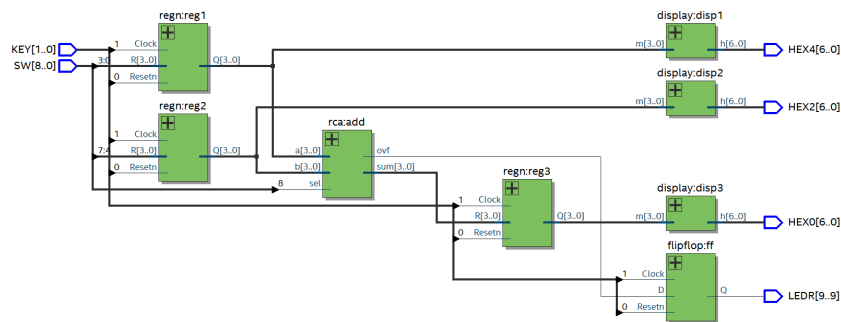
c) Synthesis:

The result, shown in the picture below, is exactly what we expected.

Initially, all three displays are off. Then, at the first rising edge of the clock, the inputs compare. At the second rising edge, the output compares. Then we reset (all displays to zero) and we try to make a subtraction between the same two numbers as before: we have overflow. We reset again and tried with a third operation, this time a subtraction without overflow.



For what concerns the timing analyzer, we obtained that the maximum operating frequency of the circuit is 486,85 MHz at 0° C, and 509,94 MHz at 85° C.



Just as before, the longest path goes from reg1 to flipflop, passing through rca.

3. 16-bit RCA, CBA and CSA

a) Design entry:

We had to design three circuits: the first is just a 16-bit version of the RCA seen in point 1. The second and the third are still 16-bit adders but respectively use the carry-bypass and the carry-select methods. The aim of this was to confront their maximum working frequency and show which of the three methods is the best.

- **RCA:** the design is the same as the first 1 of this lab. We just modified it to accept as input two 16-bit signed binary values. This meant changing the top-level entity called "rca_16bit" to make it work with 16-bit values. The other file we changed is the "rca" which now calls 16 times the full_adder file. All the rest is pretty much the same code already explained before.
- **Carry-Bypass Adder:** the design starts from the one of the 16-bit RCA. As for "regn", "display", "full_adder", and "flipflop" the design is the same one we already discussed (with just a slight improvement in the "full_adder" which now uses one less signal). Our top-level entity is "cba_16bit". It calls the "regn", the "flip flop", and 4 times the "display" in the same manner as already discussed. It then calls the "cba" which takes as inputs the two 16-bit signed signals (A, B) to be summed and gives as output the overflow (ovf) in std_logic and the 16-bit signed result (S). The "cba" file is the core of the carry bypass adder and uses other than the already described "full_adder" a "mux". The "mux" is the same one we've already used in the previous lab and takes two one-bit values and with a selection value chooses between these two.
The "cba" evaluate first the propagate pi value for every bit as $A(i) \text{ XOR } B(i)$. Then we give to the selection value of the mux (sel(i)) for every group of four bits the result of an AND evaluation to decide if the carry can be propagated or not. We can now just call the "full_adder" to perform the additions and the "mux" to decide if we need to propagate (for every group of 4 bits) the carry or take the new one.
- **Carry-Select Adder:** the design starts from the one of the 16-bit RCA. As for

“regn”, “display”, “full_adder”, and “flipflop” the design are the same ones we already discussed. Our top-level entity is “csa_16bit”. It calls the “regn”, the “flip flop”, and 4 times the “display” in the same manner as already discussed. Also the “csa” has the same ports and is called just as we previously described the “cba”. The “csa” uses two RCA’s whose structure we have already described. The only difference is that “RCAovf” gives us the overflow value instead of the output carry. We proceed to call for every group of 4 bits the “RCA” two times: one with the input carry equal to 0 and the other equal to 1 (except for the first group which is called only once as we already know the input carry is 0). Then we call the “MUX” which has as input the two sums (s1 and s0 the first evaluated with input carry 1 and the other with 0) and the two output carries and as a selection the output carry of the previous step. The “MUX” decides which of the two sum we have to choose and gives as output the chosen sum and its output carry. For the last group of bits we use the “RCAovf” as we don’t need the carry but the overflow.

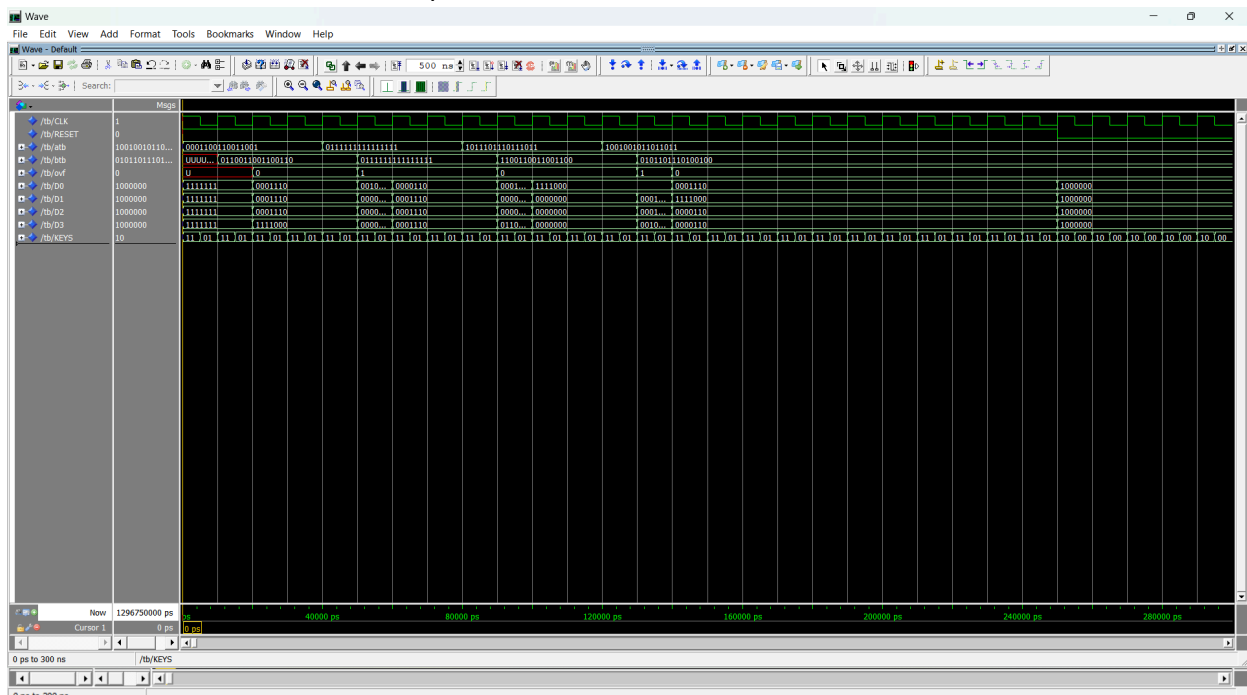
b) Functional simulation:

The testbench “tb” is the same for all the 3 versions of the 16-bit adder as the input and output are the same. Our top-level entity is “cba_16bit”. It calls the “regn”, the “flip flop”, and 4 times the “display” in the same manner as already discussed.

We have the port of the top-level entity above described. We generated a clock cycle with a period of 10ns and we reset after 250ns. We then took some values for A and B to try it and we obtained the expected results as shown in the pictures below (in order: rca, cba, csa).

c) Synthesis:

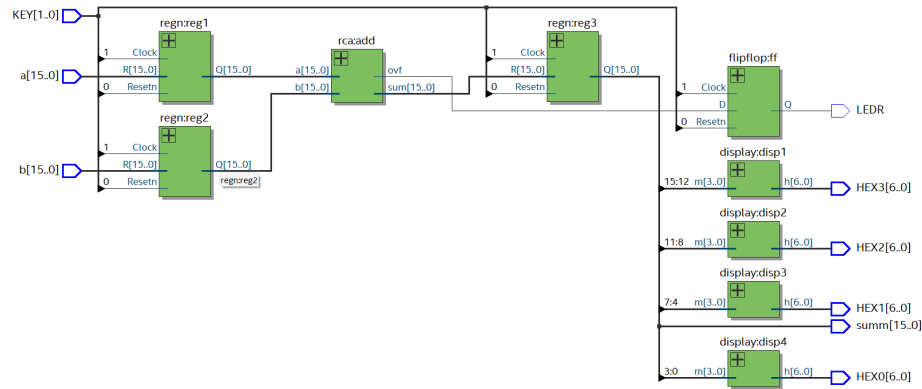
We obtained the results expected.



For what concerns the timing analyzer:

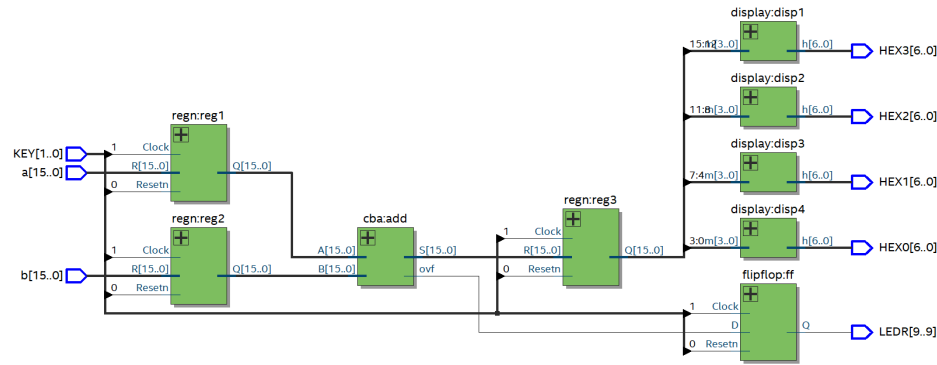
	0 °C	85 °C
RCA	294,99 MHz	287,94 MHz
CBA	246,06 MHz	243,01 MHz
CSA	393,86 MHz	389,86 MHz

RCA:



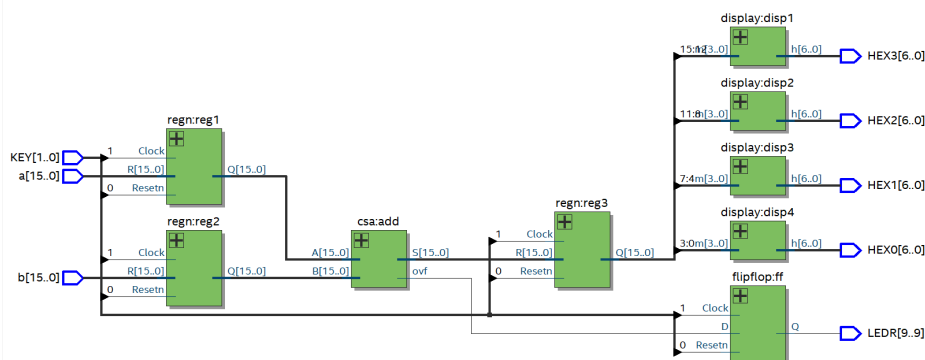
Worst-case delay paths: from reg1 to flipflop ot to reg3, through rca.

CBA:



Worst-case delay paths: from reg1 to flipflop or to reg3, through cba.

CSA:



Worst-case delay paths: from reg1 to reg3, through csa.

4. Multiplier

a) Design entry:

The following circuit is a multiplier that receives in input two 4-bit unsigned binary numbers from the switches and performs a multiplication between them. The two numbers and the result are shown on four 7-segment displays in hexadecimal format.

The top level entity is the “es4.vhd” file which takes the inputs from 8 switches and calls the “multiplier” and the “display” which respectively perform the multiplication and show the numbers on the 7-segment displays. We used a structural architecture for this file: assigning to two 4-bit signals (A and B) the two inputs taking the switches from 3 to 0 for A and 7 to 4 for B. We then called the “multiplier” and the “display”:

- as for the “display” is the same file already used in the problems above (see 1a and 2a);

				a_3	a_2	a_1	a_0
				b_3	b_2	b_1	b_0
				a_3b_0	a_2b_0	a_1b_0	a_0b_0
			a_3b_1	a_2b_1	a_1b_1	a_0b_1	
		a_3b_2	a_2b_2	a_1b_2	a_0b_2		
	a_3b_3	a_2b_3	a_1b_3	a_0b_3			
p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0

c) Implementation

Figure 5 - Multiplication of binary numbers.

- the “multiplier” takes in input A and B (unsigned) and gives in output an 8-bit unsigned number (RES) which is our result. The multiplication is performed (as

requested by the delivery of the exercise) with an array multiplier whose structure is also given in the text of the exercise.

As we already knew the structure and we needed to perform a lot of AND operations we used a mix of structural and dataflow architecture. The multiplication can be fragmented into three rows in each row we have the A input and the B input exploited by a series of AND gates as shown in the two pictures. After the AND gates, we give what we obtained into an RCA (composed of 4 full adders). The output goes either directly into the RES vector or into the next row. The structure is shown below.

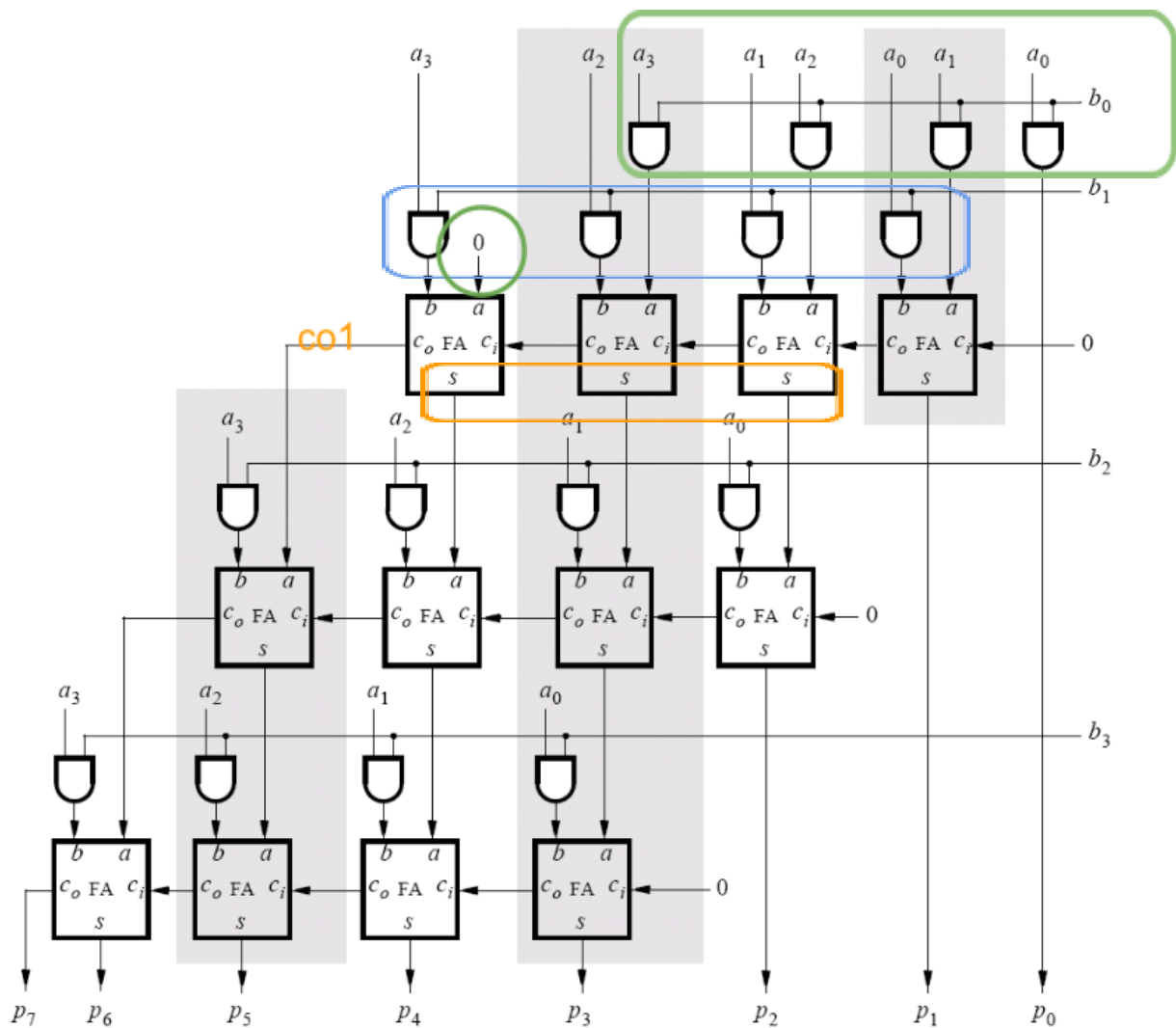


Figure 6 - An array multiplier circuit.

As an example of how we built a row, we take the first one the colors show what each part of the code is describing:

```

ROW1A(0) <= A(1) AND B(0);
ROW1A(1) <= A(2) AND B(0);
ROW1A(2) <= A(3) AND B(0);
ROW1A(3) <= '0';
ROW1B(0) <= A(0) AND B(1);

```

```

ROW1B(1) <= A(1) AND B(1);
ROW1B(2) <= A(2) AND B(1);
ROW1B(3) <= A(3) AND B(1);
ROW2A <= co1 & sum1(3 downto 1);
...
RES <= co3 & sum3 & sum2(0) & sum1(0) & (A(0) AND B(0));

```

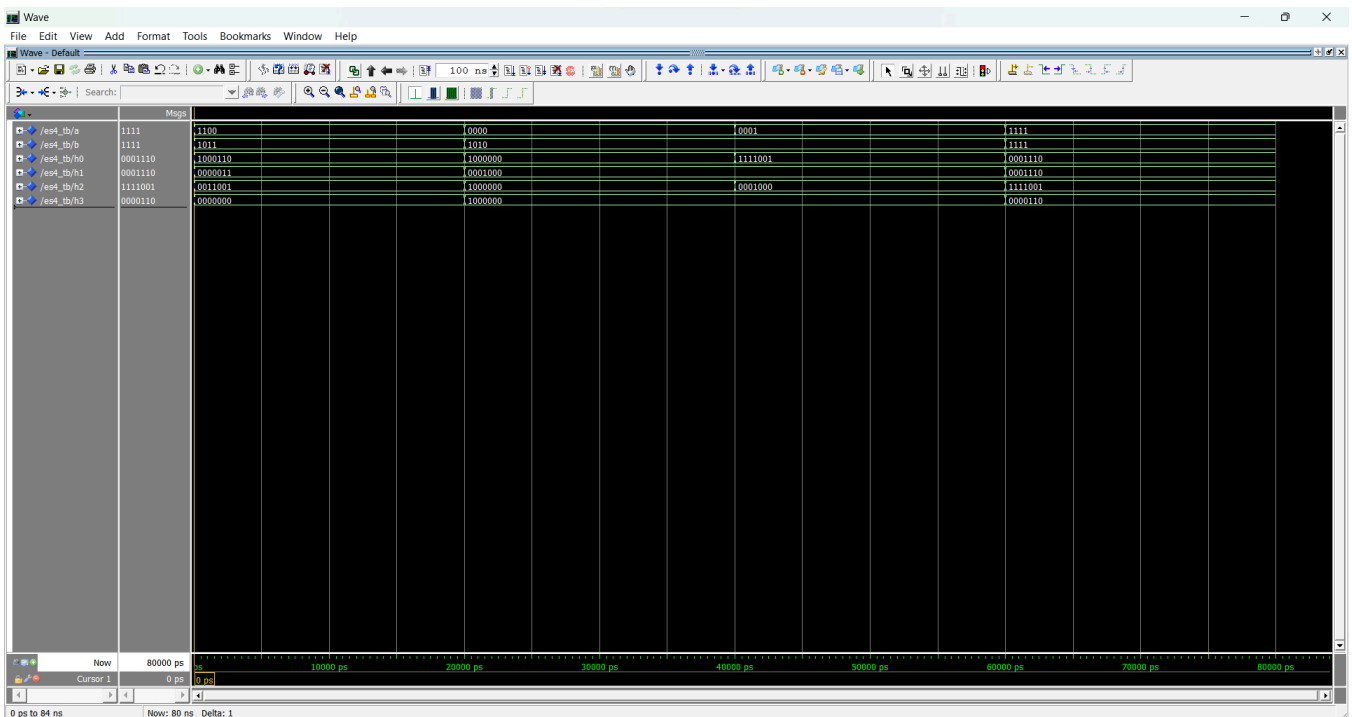
The carry of the first RCA (co1) and the 3 MSB bits of the result (sum1(3 downto 1)) are given to the next row (ROW2A), while the LSB of the result of the RCA (sum1(0)) and the output of the first AND gate (A(0) AND B(0)) are given directly in the result. We proceeded like this for all the rows and then built the result RES as a concatenation of the various results as shown above.

- The “full_adder” and the “RCA” are the same ones we already used in this lab and previous labs.

b) Functional simulation:

To test the circuit we used a testbench with a series of multiplication. We chose some inputs and showed what parts of the 7-segment displays would activate.

All the results we got from the ModelSim simulation are coherent with what we were expecting, as shown in the picture below:



c) Synthesis:

We first gave as input 1100 (in hexadecimal C) and 1011 (in hexadecimal B) with as a result in hexadecimal 84. We see that h3 and h2 are displaying 8 and 4 while h0 and h1 are displaying c and b.

We then gave as input 0000 (in hexadecimal 0) and 1010 (in hexadecimal A) with as a result in hexadecimal 00. We see that h3 and h2 are displaying 0 and 0 while h0 and h1 are displaying 0 and a.

We then gave as input 0001 (in hexadecimal 1) and 1010 (in hexadecimal B) with as a result in hexadecimal 0A. We see that h3 and h2 are displaying 0 and a while h0 and h1 are displaying 1 and b.

We then gave as input 1111 (in hexadecimal F) and 1111 (in hexadecimal F) with as a result in hexadecimal E1. We see that h3 and h2 are displaying E and 1 while h0 and h1 are displaying F and F.

For what concerns the timing analyzer, we obtained that the maximum operating frequency of the circuit is 486,85 MHz at 0° C, and 509,94 MHz at 85° C.